

CMPE679: Deep Learning
Homework 3
Fine-Tuning Transformers for Text and Vision

Gloria Mbaka
Performed: 2/23/2026
Submitted: 2/24/2026

Instructor: Dr. Liu, Dongfang
TA: Huang, Renzhong and Renjen, Ishan

By submitting this report, I attest that I neither have given nor have received any unauthorized assistance (including writing, collecting data, plotting figures, tables, or graphs, or using previous student reports as a reference), and I further acknowledge that giving or receiving such assistance will result in a failing grade for this course.

Your Signature: CGM

1. Introduction

This homework investigated fine-tuning pretrained transformer architectures for both natural language processing and computer vision tasks. Unlike Homework 1, which involved constructing a neural network from scratch, this assignment focused on transfer learning using large pretrained models. The objective was to implement a reusable fine-tuning pipeline applicable to both language models and vision transformers, and to analyze how hyperparameter choices affect convergence behavior, generalization performance, and computational cost.

Two models were studied:

- **DistilBERT** for sentiment classification on the IMDB dataset
- **Vision Transformer (ViT)** for image classification on CIFAR-10

For each modality, two different hyperparameter configurations were evaluated. Training and validation loss curves, accuracy curves, epoch time, and GPU memory usage were analyzed to compare stability and efficiency.

2. Dataset and Problem Definition

2.1 Text Modality: IMDB Sentiment Classification

The IMDB dataset consists of movie reviews labeled as positive or negative. Each review is represented as a sequence of tokens.

Given input sequence

$$\mathbf{x} = (t_1, t_2, \dots, t_n)$$

the model predicts

$$\hat{y} \in \{0,1\}$$

Tokenization was performed using the pretrained DistilBERT tokenizer, which converts raw text into token IDs and attention masks. The task was formulated as supervised binary classification.

2.2 Vision Modality: CIFAR-10 Classification

The CIFAR-10 dataset consists of 32×32 RGB images across 10 object categories. Images were resized to 224×224 to match ViT input requirements.

Given image

$$\mathbf{x} \in \mathbb{R}^{3 \times 224 \times 224}$$

the model predicts

$$\hat{y} \in \{0, 1, \dots, 9\}$$

The task was formulated as supervised multi-class classification using cross-entropy loss.

3. Model Architecture Design

3.1 DistilBERT Architecture

DistilBERT is a transformer encoder composed of multi-head self-attention layers and feedforward sublayers. For classification, a task-specific linear head was appended.

Let

$$\mathbf{h}_{cls} \in \mathbb{R}^{768}$$

denote the pooled representation of the input sequence. The classifier computes

$$\mathbf{z} = \mathbf{W}\mathbf{h}_{cls} + \mathbf{b}$$

where

$$\mathbf{W} \in \mathbb{R}^{2 \times 768}, \mathbf{b} \in \mathbb{R}^2$$

The output vector

$$\mathbf{z} \in \mathbb{R}^2$$

contains logits corresponding to the two sentiment classes.

3.2 Vision Transformer (ViT)

ViT divides the image into fixed-size patches and linearly embeds them into token vectors. A learnable classification token is prepended and passed through transformer encoder blocks.

Let

$$\mathbf{h}_{cls} \in \mathbb{R}^{768}$$

be the final representation of the classification token. The classifier computes

$$\mathbf{z} = \mathbf{W}\mathbf{h}_{cls} + \mathbf{b}$$

where

$$\mathbf{W} \in \mathbb{R}^{10 \times 768}, \mathbf{b} \in \mathbb{R}^{10}$$

The logits

$$\mathbf{z} \in \mathbb{R}^{10}$$

correspond to the ten CIFAR-10 classes.

4. Training Methodology

4.1 Loss Function

For both modalities, cross-entropy loss was used.

Given logits

$$\mathbf{z} \in \mathbb{R}^K$$

class probabilities are computed via softmax:

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$$

The loss for true label y is:

$$\mathcal{L} = -\log(p_y)$$

In PyTorch:

```
criterion = nn.CrossEntropyLoss()  
loss = criterion(outputs, labels)
```

4.2 Optimization

Model parameters were updated using Adam:

$$\theta \leftarrow \theta - \eta \frac{\partial \mathcal{L}}{\partial \theta}$$

Two hyperparameter configurations were tested per modality.

DistilBERT

1. Learning rate 2×10^{-5} , batch size 16
2. Learning rate 5×10^{-5} , batch size 32

ViT

1. Learning rate 5×10^{-5} , batch size 64
2. Learning rate 1×10^{-4} , batch size 128

Early stopping was applied for ViT based on validation loss.

5. Evaluation Methodology

After each epoch:

- Model set to evaluation mode
- Gradients disabled using `torch.no_grad()`
- Validation loss and accuracy recorded
- Epoch time and peak GPU memory measured

Accuracy was computed as:

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Samples}}$$

6. Results and Visualization

6.1 DistilBERT Results

Two hyperparameter configurations were evaluated: (1) learning rate 2×10^{-5} with batch size 16, and (2) learning rate 5×10^{-5} with batch size 32. Both settings converged quickly and achieved similar validation accuracy near 0.93. The lower learning rate with smaller batch size produced more stable validation behavior, with validation loss decreasing steadily from 0.2021 to 0.1960 and validation accuracy improving to 0.9296 by epoch 5. In contrast, the higher learning rate with larger batch size reached its best validation accuracy slightly earlier (0.9301 at epoch 4) and attained a lower minimum validation loss (0.1897 at epoch 4), but validation loss increased sharply to 0.2184 by epoch 5, indicating mild overfitting or optimization instability. In terms of compute cost, batch size 32 reduced epoch time slightly (~ 212 s vs ~ 222 s) but increased peak GPU memory substantially (~ 2349 MB vs ~ 1373 MB). Overall, the higher learning rate and larger batch accelerated early convergence but reduced stability, while the smaller learning rate provided smoother generalization.

Figures 1 and 2 show training and validation loss and accuracy for both hyperparameter configurations.

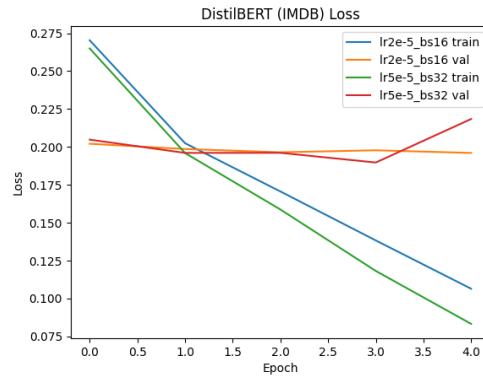


Figure 1: DistilBERT training and validation loss curves

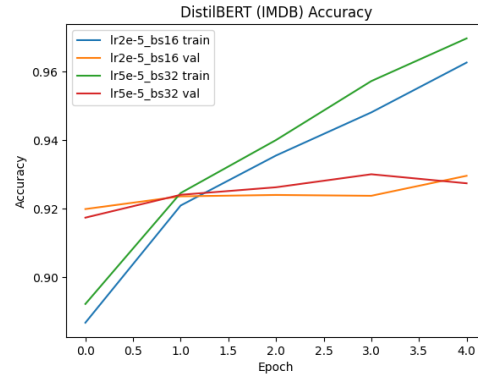


Figure 2: DistilBERT training and validation accuracy curves

The configuration with learning rate 5×10^{-5} and batch size 32 converged faster, as training loss decreased more rapidly. However, validation loss increased during the final epoch, indicating mild instability. The configuration with learning rate 2×10^{-5} and batch size 16 showed smoother validation behavior and more stable generalization.

Compute comparison:

Config	Epoch Time	Peak Memory
2e-5, 16	~222 s	~1373 MB
5e-5, 32	~212 s	~2349 MB

Larger batch size improved throughput but significantly increased memory consumption.

6.2 ViT Results

The ViT model converged extremely quickly on CIFAR-10 under both hyperparameter configurations. Training loss decreased sharply within the first epoch and continued declining steadily, reaching near-zero values by epoch 3–4. Training accuracy exceeded 99%, indicating that the model has sufficient capacity to fit the dataset effectively. Validation loss revealed clearer differences between configurations. The $5e-5$ learning rate with batch size 64 achieved the lowest validation loss (approximately 0.0707) and maintained slightly more stable behavior before early stopping. In contrast, the higher learning rate ($1e-4$) with batch size 128 achieved strong validation accuracy but exhibited a gradual increase in validation loss after the first epoch, indicating earlier overfitting or reduced optimization stability. Interestingly, validation accuracy remained high for both configurations ($\sim 97.8\%$), even as validation loss increased slightly. This suggests that while classification accuracy remained stable, the higher learning rate caused the model to become more overconfident in its predictions. Overall, ViT demonstrated faster convergence and higher absolute accuracy compared to the text model but also showed sensitivity to learning rate and earlier signs of overfitting. Figures 3 and 4 show ViT loss and accuracy curves.

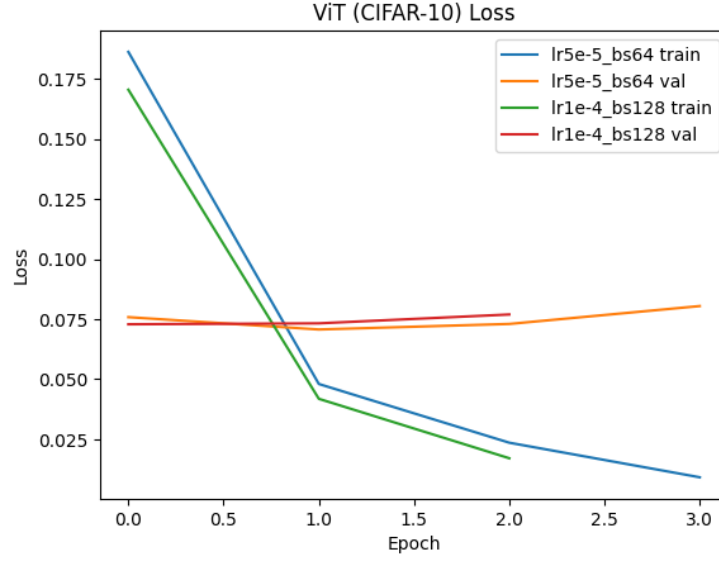


Figure 3: ViT training and validation loss curves

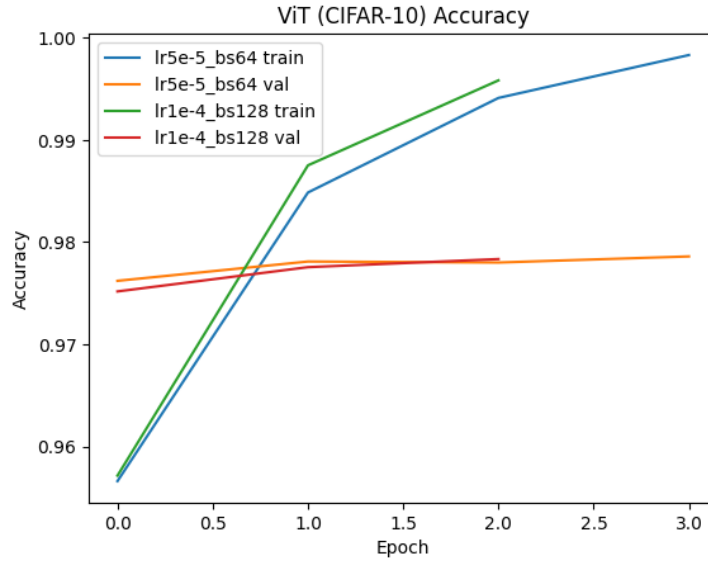


Figure 4: ViT training and validation accuracy curves

Both configurations converged rapidly, achieving validation accuracy near 97.8%. The configuration with learning rate 5×10^{-5} and batch size 64 achieved the lowest validation loss (~ 0.0707) and demonstrated slightly better stability. The higher learning rate configuration (1×10^{-4} , batch size 128) converged quickly but showed earlier overfitting, as validation loss increased after the first epoch.

Compute comparison:

Config	Epoch Time	Peak Memory
5e-5, 64	~176 s	~1942 MB
1e-4, 128	~173 s	~3387 MB

Again, larger batch size slightly reduced training time but nearly doubled memory usage.

7. Discussion and Lessons Learned

This assignment demonstrated that a unified fine-tuning pipeline can be reused across modalities. While both DistilBERT and ViT benefited from pretrained representations, their convergence characteristics differed.

DistilBERT converged more gradually and exhibited milder overfitting behavior. ViT converged significantly faster and achieved higher validation accuracy but was more sensitive to learning rate increases.

Across both modalities:

- Larger learning rates accelerated convergence
- Larger batch sizes reduced epoch time
- Larger batch sizes substantially increased GPU memory usage
- Higher learning rates introduced mild instability in validation loss

These findings highlight the importance of balancing optimization speed and generalization stability when fine-tuning large pretrained models.

8. Conclusion

This assignment provided hands-on experience with fine-tuning transformer models for both text and vision tasks. A reusable training framework was successfully applied across modalities. Experimental analysis demonstrated clear trade-offs between convergence speed, stability, and computational cost. While aggressive hyperparameters improved early convergence and throughput, they increased memory usage and slightly reduced validation stability. Overall, this homework strengthened understanding of transfer learning, hyperparameter sensitivity, and transformer-based model evaluation.

9. AI Usage Disclosure

Artificial intelligence tools were used to clarify theoretical concepts, assist with debugging, and refine report organization. All experiments, code implementations, plots, and numerical results were generated independently by the student. AI assistance was limited to conceptual guidance and editorial refinement.

References

1. Devlin, J., et al. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.
2. Dosovitskiy, A., et al. (2021). An Image is Worth 16×16 Words.
3. Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.
4. Paszke, A., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library.